

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

CNN vs Fully Connected Neural Network

A **Convolutional Neural Network (CNN)** is specifically designed to process image data, whereas a **traditional Fully Connected Neural Network (FCNN)** treats the input image mainly as a flat vector. For medical X-ray image classification, CNNs are generally more suitable because they can automatically learn important **spatial and visual features**.

1. Architectural Design

Fully Connected Neural Network:

- The X-ray image is first flattened into a one-dimensional vector.
- Every neuron in one layer is connected to every neuron in the next layer.
- It does not naturally preserve the spatial relationship between neighboring pixels.

CNN:

- Uses a sequence of convolutional layers, pooling layers, and fully connected layers.

- It processes the image as a 2D/3D structure.
- Spatial relationships between pixels are preserved during feature extraction.

2. Motivation

The main motivation for CNNs is to efficiently recognize patterns such as edges, textures, shapes, and anatomical structures in images.

For example, in an X-ray:

- Early layers may detect edges.
- Middle layers may detect shapes and textures.
- Deeper layers can identify complex patterns associated with abnormalities or diseases.

3. Spatial Feature Extraction

A traditional FCNN loses much of the spatial information when the image is flattened.

CNNs preserve spatial information by applying filters over small regions of the image. This allows the network to learn where important features occur and how neighboring pixels are related.

4. Role of Convolutional Layers

Convolutional layers are the main feature-extraction component of a CNN.

They:

- Apply learnable filters to the input image.
- Detect features such as edges, curves, textures, and patterns.
- Produce feature maps.
- Learn useful features automatically during training.

For X-ray classification, convolutional layers can learn features related to structures such as lungs, bones, and abnormal regions.

5. Role of Filters

A filter (kernel) is a small matrix that slides across the image.

Different filters learn different patterns. For example:

- One filter may detect horizontal edges.
- Another may detect vertical edges.
- Deeper filters can learn more complex patterns.

Thus, CNNs automatically learn useful image features instead of requiring manual feature extraction.

6. Role of Pooling Layers

Pooling layers reduce the spatial dimensions of feature maps.

Common types include:

- **Max Pooling** – selects the maximum value from a region.
- **Average Pooling** – calculates the average value.

Pooling:

- Reduces the amount of computation.
- Reduces the number of parameters.
- Helps prevent overfitting.
- Makes the network less sensitive to small changes in image position.

7. Number of Parameters

FCNNs generally require a **very large number of parameters** for high-resolution images because every input pixel can connect to every neuron.

CNNs use **local connectivity and weight sharing**. The same filter is reused across different parts of the image, greatly reducing the number of parameters.

For example, a 224×224 image contains **50,176 pixels** per channel. Connecting all these pixels directly to thousands of neurons would create a very large number of weights.

8. Computational Complexity

Feature	Fully Connected Neural Network	CNN
Input	Flattened image	Image structure preserved
Feature extraction	Mainly learned through dense connections	Convolutional filters
Spatial information	Mostly lost	Preserved
Parameters	Very high	Relatively lower
Computation	High for large images	More efficient
Image suitability	Poor to moderate	Excellent

9. Ability to Handle Image Data

FCNNs can classify images, but they become inefficient as image size increases. They also do not naturally exploit local patterns.

CNNs are specifically designed for image data. They can recognize important features even when the feature appears at different locations in the image.

10. Why CNNs Are Better for X-ray Classification

CNNs are more suitable for medical X-ray classification because they:

1. Preserve spatial relationships between pixels.

2. Automatically extract important image features.
3. Use fewer parameters through weight sharing.
4. Reduce computational requirements compared with a large FCNN.
5. Detect features at different levels of complexity.
6. Provide better performance for image recognition and classification tasks.

Conclusion

A Fully Connected Neural Network can classify X-ray images but requires flattening the images and usually has a large number of parameters. A CNN, on the other hand, uses convolutional filters and pooling to automatically extract spatial features while reducing computational complexity. Therefore, CNNs are more efficient and effective for medical X-ray image classification, making them the preferred deep learning architecture for applications such as pneumonia, tuberculosis, and other abnormality detection.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Different CNN Layers and Filters

A **Convolutional Neural Network (CNN)** is widely used for industrial defect detection because it can automatically learn visual features from product images. Different CNN layers perform different functions, from basic feature extraction to final classification.

1. Convolutional Layers

Convolutional layers are the main **feature extraction layers** of a CNN.

- They use small learnable **filters (kernels)** that slide over the input image.
- They detect features such as edges, corners, textures, cracks, and patterns.
- They preserve the spatial arrangement of features.
- The same filter is reused across the image, reducing the number of parameters and computation.

Example: A filter may detect a crack or scratch on a manufactured product.

2. Pooling Layers

Pooling layers reduce the spatial size of feature maps.

Common types are:

- **Max pooling** – selects the maximum value from a region.
- **Average pooling** – calculates the average value.

Functions:

- Reduce computational requirements.
- Reduce the number of feature-map values.
- Help prevent overfitting.
- Retain important features while removing less important information.

For defect detection, pooling helps retain strong features such as prominent cracks while reducing unnecessary details.

3. Activation Layers

Activation functions introduce **non-linearity** into the CNN.

A commonly used activation function is **ReLU (Rectified Linear Unit)**:

$$\text{ReLU}(x) = \max(0, x)$$

Functions:

- Allows the network to learn complex patterns.
- Converts negative values to zero.
- Helps CNNs learn complicated defect characteristics.
- Improves training efficiency.

Without activation functions, multiple CNN layers would behave approximately like a single linear transformation.

4. Fully Connected Layers

Fully connected layers are generally used toward the end of the CNN.

- They receive the important features extracted by previous layers.
- They combine these features for classification.
- The final layer can use **Softmax** for multiple defect classes or **Sigmoid** for binary classification.

For example, the system may classify an image as:

No Defect | Crack | Scratch | Dent | Surface Damage

Comparison of CNN Layers

Layer	Main Function	Spatial Information	Computational Requirement
Convolution	Feature extraction	Preserves spatial structure	Moderate
Pooling	Dimension reduction	Reduces spatial size	Low
Activation	Adds non-linearity	Maintains feature-map structure	Low
Fully Connected	Classification	Mostly loses spatial structure	High

Early-Layer Filters vs Deeper-Layer Filters

CNN filters learn different types of features at different depths.

Early-Layer Filters

The first few convolutional layers learn **simple and low-level features**.

They commonly detect:

- Edges
- Lines
- Corners
- Basic textures
- Simple color/intensity changes

For industrial products, early filters may detect the boundaries of scratches, cracks, or surface irregularities.

Deeper-Layer Filters

Deeper convolutional layers combine simple features to learn **complex and high-level features**.

They can detect:

- Complex textures
- Shapes
- Patterns
- Crack structures
- Specific defect characteristics

For example, several simple edge features detected in early layers can be combined by deeper layers to identify a **long surface crack**.

Comparison

Feature	Early Filters	Deeper Filters
Feature level	Low-level	High-level
Detects	Edges, lines, corners	Shapes, textures, defects
Complexity	Simple	Complex
Information	Local	More abstract
Example	Edge of a crack	Complete crack pattern

How These Layers Improve Defect Detection

The CNN works in a hierarchical manner:

Input Product Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Defect Classification

1. **Convolutional layers** extract useful visual features.
2. **Activation layers** allow the network to learn complex relationships.
3. **Pooling layers** reduce image dimensions and computational cost.
4. **Deeper convolutional layers** combine simple features into complex defect patterns.
5. **Fully connected layers** use the extracted features to classify the product.
6. The final output identifies whether the product is defective and, if required, determines the **type of defect**.

Conclusion

Convolutional, pooling, activation, and fully connected layers work together to create an effective defect-detection system. **Early filters identify simple features such as edges and textures, while deeper filters identify complex defect patterns.** Pooling reduces computational requirements, activation provides non-linearity, and fully connected layers perform final classification. Therefore, CNNs can accurately and automatically detect different types of industrial product defects.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

Parameter Sharing vs Independent Weights

In a smart surveillance system, thousands of high-resolution images must be processed efficiently. **CNNs use parameter sharing**, while fully connected networks use **independent**

weights for different connections. Parameter sharing makes CNNs more suitable for image-processing applications.

1. Parameter Sharing in CNNs

In CNNs, the same **filter weights** are reused across different regions of an image. For example, a 3×3 filter has only 9 weights, and these weights are applied throughout the image. This allows the filter to detect the same feature at different locations.

2. Independent Weights in Fully Connected Networks

In a fully connected network, every input pixel is connected to neurons using separate weights. For a high-resolution image, this results in a very large number of parameters, increasing memory and computational requirements.

3. Comparison

Factor	CNN	Fully Connected Network
Parameters	Fewer due to sharing	Very large
Memory	Lower	Higher
Computation	More efficient	More expensive
Feature detection	Location-independent	More location-specific
Image processing	Highly suitable	Less suitable

4. Feature Detection

Parameter sharing allows CNNs to detect common features such as **edges, textures, shapes, and objects** wherever they appear in an image. For example, the same filter can detect a vehicle whether it appears on the left or right side of a surveillance image.

5. Translation-Related Invariance

CNNs provide **translation equivariance**, meaning that when an object moves in an image, its feature response also moves accordingly. Pooling and deeper CNN layers can provide some translation tolerance, helping recognize objects even when their exact position changes.

6. Learning Efficiency

Since the same weights are used at many locations, CNNs need to learn fewer parameters. The filters receive information from multiple image regions, allowing them to learn useful features more efficiently and reducing the risk of overfitting.

7. Importance for Large-Scale Applications

Smart surveillance systems process a large number of images containing **people, vehicles, faces, and suspicious objects**. Using independent weights would require excessive memory and computation. Parameter sharing makes CNNs faster, more scalable, and more practical.

Conclusion

Parameter sharing significantly reduces the **number of parameters, memory usage, and computational cost** of CNNs. It also allows the same visual features to be detected at different locations. Therefore, parameter sharing is a key reason why CNNs are preferred for **large-scale image-processing and surveillance applications**.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

AlexNet vs ResNet

AlexNet and **ResNet** are important CNN architectures used for image classification. AlexNet introduced deep CNNs to large-scale image recognition, while ResNet improved deep network training using **residual connections**.

1. Comparison of AlexNet and ResNet

Feature	AlexNet	ResNet
Network depth	Relatively shallow, 8 learnable layers	Much deeper, e.g., ResNet-50/101/152
Layer organization	Sequential convolution and fully connected layers	Convolutional layers with residual/skip connections
Filters	Uses larger early filters such as 11×11	Mainly uses smaller 3×3 filters
Parameters	About 60 million	Varies by version; ResNet-50 has about 25.6 million
Feature extraction	Good for basic and intermediate features	Excellent for complex hierarchical features
Training difficulty	Easier than very deep networks	Easier than similar-depth plain networks
Vanishing gradient	More limited due to lower depth	Strongly reduced using skip connections
Computational cost	Lower for basic models	Higher, especially for deeper versions
Classification performance	Good	Generally better on complex tasks

2. Network Depth

AlexNet has **8 learnable layers**, including convolutional and fully connected layers. ResNet is available in different depths such as **ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152**.

The greater depth of ResNet allows it to learn more complex image features.

3. Convolutional Filters

AlexNet uses relatively large filters in its early layers, such as **11×11 and 5×5 filters**. ResNet mainly uses **3×3 filters**, which are efficient and allow the network to build complex features through multiple layers.

4. Parameter Requirements

AlexNet has around **60 million parameters**, mainly because of its large fully connected layers.

ResNet-50 has around **25.6 million parameters**, despite being much deeper. Its efficient convolutional design and bottleneck blocks help reduce parameter requirements.

5. Feature Extraction

AlexNet can learn features such as:

- Edges
- Textures
- Shapes
- Simple object patterns

ResNet's deeper structure can learn more complex hierarchical features, making it better for challenging image recognition tasks involving many object categories and variations.

6. Training Difficulty and Vanishing Gradients

As networks become deeper, **vanishing gradients** can make training difficult. ResNet solves this problem using **skip connections**, which allow information and gradients to pass directly through layers.

A residual block can be represented as:

$$\text{Output} = F(x) + x$$

where x is the original input and $F(x)$ represents the learned transformation.

7. Computational Cost

AlexNet is generally less computationally demanding because it is shallower. ResNet requires more computation because of its greater depth, although its architecture is more parameter-efficient than its depth might suggest.

8. Classification Performance

AlexNet provides good performance for basic image classification. However, ResNet generally achieves **higher accuracy and better feature representation** on complex image recognition tasks because it can learn deeper and more meaningful features.

9. Recommendation

For a **complex image recognition application, ResNet is the better choice.**

Reasons:

- Supports very deep networks.
- Reduces the vanishing-gradient problem.
- Extracts more complex features.
- Uses efficient residual blocks.
- Provides strong classification performance.
- Can be scaled using different ResNet versions.

Conclusion

AlexNet is a simpler and historically important CNN architecture, while ResNet provides a deeper and more efficient approach using **residual/skip connections**. Although ResNet can require greater computational resources, its ability to learn complex features and overcome the vanishing-gradient problem makes **ResNet more suitable for complex image classification applications**.

Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

CNN Regularization for Plant Disease Classification

A CNN used for plant disease classification may achieve very high training accuracy but poor performance on unseen leaf images. This indicates **overfitting**, where the model learns the training images too closely instead of learning general disease patterns.

1. CNN Without Regularization

A CNN without regularization can memorize training images.

- Training accuracy becomes very high.
- Validation/test accuracy remains low.
- Sensitive to variations in lighting, leaf position, and background.

- Generalization to new images is poor.
- Training may become unstable in deeper networks.

2. CNN With Regularization

Using **dropout**, **data augmentation**, and **batch normalization** helps the CNN learn more general and reliable features.

Factor	Without Regularization	With Regularization
Overfitting	High	Reduced
Generalization	Poor	Better
Training stability	Lower	Higher
Computational cost	Lower	Slightly higher
Performance on new images	Poor	Better

3. Dropout

Dropout randomly deactivates some neurons during training.

For example, with a dropout rate of 0.5, approximately half of the selected neurons are temporarily ignored during each training step.

Benefits:

- Prevents neurons from depending too much on each other.
- Reduces overfitting.
- Improves generalization.

4. Data Augmentation

Data augmentation creates modified versions of existing leaf images using transformations such as:

- Rotation
- Flipping
- Zooming
- Cropping
- Small brightness changes

This increases the variety of training data and helps the model recognize diseases under different real-world conditions.

For plant disease classification, data augmentation is particularly important because leaf images can vary in orientation, size, lighting, and background.

5. Batch Normalization

Batch normalization normalizes the activations of a network during training.

It helps:

- Stabilize training.
- Speed up convergence.
- Allow more effective learning.
- Reduce sensitivity to initialization.

It can also provide a small regularization effect, although its main purpose is **training stabilization**, not replacing dropout or augmentation.

6. Computational Requirements

A CNN without regularization requires less computation because it performs fewer operations.

A regularized CNN requires slightly more computation, especially during data augmentation and training. However, this additional cost is usually worthwhile because it produces a model that performs better on unseen images.

7. Recommended Approach

For reliable plant disease classification, the organization should use **data augmentation + batch normalization + suitable dropout**.

A typical structure can be:

Leaf Image → Data Augmentation → Convolution → Batch Normalization → ReLU → Pooling → Convolution → Batch Normalization → Dropout → Classification

8. Justification

- **Data augmentation** improves the diversity of training images.
- **Batch normalization** improves training stability and convergence.
- **Dropout** reduces over-dependence on particular neurons.
- Together, they reduce overfitting and improve performance on unseen leaf images.

Conclusion

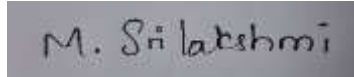
The poor performance on unseen images indicates that the original CNN is **overfitting**. Therefore, a CNN using **data augmentation, batch normalization, and dropout** is more suitable. Among these, **data augmentation is especially important** for real-world plant disease classification because it exposes the model to variations in leaf orientation, lighting, and appearance. Combining all three techniques provides better **generalization, stability, and reliable disease classification**.

Code of Conduct:

I **M. Srilakshmi (192525404)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature is written in a cursive style and reads "M. Srilakshmi".

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand